Spring Rolls: Collaborative Filtering

Justina Dai [*] **Julius Wendelin Wemhöner** [*] **Samuel Garcin** [*] **Nico Kobler** [*]

Abstract

In Collaborative Filtering (CF), accurately predicting user preferences through explicit feedback is crucial for personalized recommendation, yet it remains challenging due to data sparsity. Implicit feedback offers a more abundant alternative and provides a complementary perspective on user behaviour. In this report, a new model based on a stacking ensemble with 47 base CF predictors and a hand-crafted context vector of user and item statistics, combined by a gradient-boosted meta-learner, is introduced to predict ratings from explicit and implicit feedback. This new model is compared with 18 existing methods covering neighbourhood-based, matrix factorization, and deep learning approaches on the comparatively dense CIL dataset, indicating promising performance. In addition, ablation studies are conducted to systematically analyze the contribution of individual components of the ensemble. To study the contribution of implicit feedback in real-world applications with sparse data, further testing on datasets with higher sparsity is needed. **Code availability:** `https://github.com/wendelinwemhoener/cil-project`

1. Introduction

Popular Web applications such as Amazon, Facebook, and Netflix rely heavily on automatic personalization to recommend relevant content to a large number of users (Smith & Linden, 2017; Gomez-Uribe & Hunt, 2016). However, data sparsity remains a challenge because most users only interact with a small fraction of available items in these web applications.

Collaborative Filtering (CF) is a widely adopted technique in recommendation systems that predicts user preferences based on past user-item interactions (Su & Khoshgoftaar, 2009). The field was popularized by the Netflix Prize (2006–2009), a famous public competition to predict user ratings for films, where almost 99% of potential user-film pairs had no ratings (Bennett & Lanning, 2007). Three key aspects learned from it are (Bell & Koren, 2007; Koren et al., 2009a; Koren & Bell, 2011): (1) ensembles consistently outperformed individual models by combining them complementarily; (2) the matrix factorization approach, which represents users and items as latent vectors in a shared low-dimensional space, showed consistently excellent performance; and (3) implicit feedback, e.g. the watched movies, is also important as it provided complementary but equally important information to explicit feedback on the highly sparse Netflix dataset.

Recent works have increasingly focused on implicit feedback, such as clicks, views, and purchases, rather than explicit feedback, as they are more abundant and easy to collect (He et al., 2017; Steck, 2019). However, it is noisier than explicit feedback, as it only indirectly reflects user satisfaction. Explicit feedback, though sparser, contains more direct information. The complementary nature of implicit and explicit feedback motivated our approach, which jointly leverages the advantages of both kinds of inputs.

In this work, we introduce an ensemble model which combines implicit and explicit feedback through a gradient-boosted meta-learner, augmented with a hand-crafted context vector of user and item statistics. Our contributions are three-fold:

- A stacking ensemble is introduced which combines 47 base CF predictors and a hand-crafted context vector through a Histogram-based Gradient Boosting Regressor (HGBR) meta-learner.
- We compare the performance of our new model against 18 existing methods based on various recommendation paradigms.
- We perform ablation studies to analyze the contribution of individual components of the ensemble, including a negative finding on the context vector.

2. Models and Methods

2.1. Problem statement

Consider m users $\mathcal{U}$, n items $\mathcal{I}$, and two kinds of feedback. Explicit feedback is given by ratings $r_{u,i} \in \{1, \dots, 5\}$ on a subset $\mathcal{K} \subseteq \mathcal{U} \times \mathcal{I}$ of user–item pairs. Implicit feedback is given by a wishlist $\mathcal{W} \subseteq \mathcal{U} \times \mathcal{I}$ of user–item pairs the

[*]Equal contribution . Correspondence to: Justina Dai <justdai@student.ethz.ch>.

Computational Intelligence Lab FS2026, ETH Zuerich

user has wishlisted. The task is to predict $r_{u,i}$ for pairs $(u, i) \notin \mathcal{K}$.

2.2. Model architecture

As illustrated in Figure 1, our model extends standard collaborative filtering ensembles by augmenting base predictors from implicit and explicit feedback with a hand-crafted context vector.

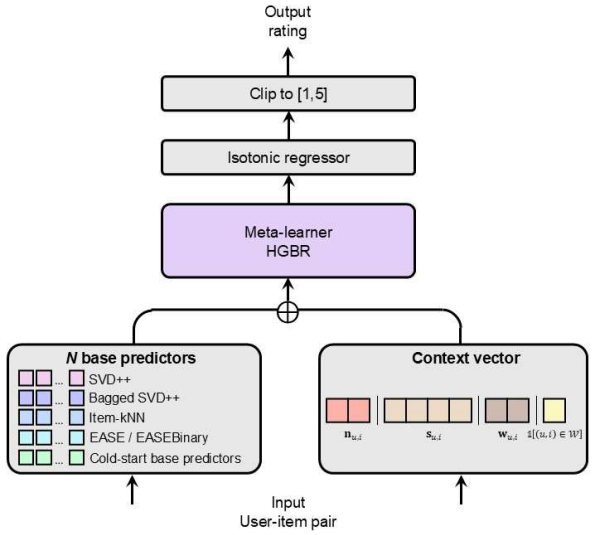


Figure 1. Our ensemble predicts ratings as follows: (1) $N = 47$ base predictors (color-coded by class) score the input user–item pair; (2) their scores are concatenated with a context vector; (3) an HGBR meta-learner produces the final prediction, calibrated and clipped to $[1, 5]$.

Each user–item pair (u, i) is scored by $N = 47$ base predictors $\{f_n\}_{n=1}^{N}$, grouped into five base predictor classes (Appendix B):

- **SVD++** (Koren, 2008): a matrix factorization model which uses both explicit ratings and implicit wishlist feedback, instantiated with varying rank, regularization, learning rate, and random seed.
- **Bagged SVD++** (Breiman, 1996): bootstrap-sampled SVD++ instances with different subsampling fractions and seeds to reduce prediction variance.
- **Item-kNN** (Sarwar et al., 2001): item-item cosine similarity models, instantiated with varying neighbourhood sizes and shrinkage.
- **EASE** (Steck, 2019) and **EASEBinary**: linear autoencoders on the explicit rating matrix and on a binary interaction matrix combining ratings and wishlist entries, respectively.
- **Cold-start base predictors**: IterSVD, SVD, and a bias-only model for robustness on sparse users and items.

In parallel, the model extracts a 9-dimensional context vector

$$\phi(u, i) = \big[\, \mathbf{n}_{u,i},\ \mathbf{s}_{u,i},\ \mathbf{w}_{u,i},\ \mathbb{1}[(u, i) \in \mathcal{W}] \,\big] \in \mathbb{R}^9,$$

where $\mathbf{n}_{u,i}, \mathbf{w}_{u,i}$ are log rating and wishlist counts, $\mathbf{s}_{u,i}$ rating statistics, and $\mathcal{W}$ the wishlist set. The base-predictor scores and context vector are concatenated into a stacked representation

$$z_{u,i} = [f_1(u, i), \ldots, f_N(u, i), \phi(u, i)],$$

on which we train a Histogram-based Gradient Boosting Regressor (HGBR) $\mathcal{M}$ as the meta-learner. Its output is calibrated by an isotonic regressor g and clipped to the rating range:

$$\hat{r}_{u,i} = \mathrm{clip}_{[1,5]}\big(g \circ \mathcal{M}(z_{u,i})\big).$$

2.3. Training

Training proceeds in three stages to prevent leakage across base predictors, meta-learner, and calibrator: (i) base predictors are first fit on the training pairs $\mathcal{K}$ using cross-validation, producing out-of-fold scores; (ii) the training pairs are then partitioned 80/20 into $\mathcal{K}_S$ and $\mathcal{K}_C$, with the meta-learner fit on the scores for $\mathcal{K}_S$; and (iii) the calibrator on the meta-learner's predictions for the held-out $\mathcal{K}_C$.

Base predictors Each base predictor f_n is trained twice. First, f_n is trained with 5-fold cross-validation on $\mathcal{K}$. This produces an out-of-fold score $f_n(u, i)$ for every $(u, i) \in \mathcal{K}$, from a model not trained on that pair. These scores define the stacked representations $\{z_{u,i}\}_{(u,i)\in\mathcal{K}}$ used to train the meta-learner. Second, f_n is trained on the full training set, to score pairs $(u, i) \notin \mathcal{K}$ at inference.

Meta-learner The HGBR meta-learner $\mathcal{M}$ is fit on the stacked representations for $\mathcal{K}_S$ with early stopping, which selects the number of trees $T^\star$ and produces held-out predictions on $\mathcal{K}_C$ for calibration. For inference, $\mathcal{M}$ is refit on all training pairs $\mathcal{K}$ with $T^\star$ fixed.

Isotonic calibration The isotonic regressor g is fit on the held-out pairs $\big\{\big(\mathcal{M}(z_{u,i}),\, r_{u,i}\big)\big\}_{(u,i)\in\mathcal{K}_C}$ to correct residual miscalibration from the stacker and the clipping step.

2.4. Experimental setup

Dataset The dataset consists of 1.2 million ratings (1–5) from $9,850$ scientists (users) on 960 papers (items), with a rating density of 13%. Additionally, implicit feedback is provided as a wishlist for each scientist, with a wish density of 4%. Detailed statistics are reported in Appendix A.

Evaluation The dataset was split into 80% training and 20% test sets. Baseline hyperparameters were tuned using 5-fold cross-validation on the training set, minimizing mean

RMSE across folds (Appendix C). Our model was trained on the training set as described in Section 2.3. All models were evaluated on the held-out test set. RMSE and MAE were reported with 95% confidence intervals (CI) derived from 10,000 bootstrap resamples.

Baselines Our ensemble is compared with 18 existing methods based on various recommendation paradigms and input data (Table 1). Short descriptions of each method are provided in Appendix D.1.

Table 1. Baselines grouped by paradigm and input data. ✓ = used as input, × = not used.

Method	Ratings	Wishlist
Naive baselines		
GlobalMean	✓	×
UserMean	✓	×
ItemMean	✓	×
SlopeOne (Lemire & Maclachlan, 2007)	✓	×
Neighborhood-based		
KNNBasic	✓	×
KNNWithMeans	✓	×
KNNWithZScore	✓	×
Item-kNN (Sarwar et al., 2001)	✓	×
Matrix factorization		
SVD	✓	×
IterSVD	✓	×
ALS (Koren et al., 2009b)	✓	×
BiasedSVD (Koren et al., 2009b)	✓	×
SVD++ (Koren, 2008)	✓	✓
Neural / linear autoencoder		
EASE (Steck, 2019)	✓	×
NeuMF (He et al., 2017)	✓	×
NeuMF† (incl. wishlist)	✓	✓
DMF (Xue et al., 2017)	✓	×
DMF† (incl. wishlist)	✓	✓
Our ensemble	✓	✓

† Our own extension of the original architecture.

3. Results

Table 2 summarizes the performance of our ensemble against the baseline methods on the holdout set.

Our ensemble achieves the best results, with an RMSE of 0.8744 ± 0.0028 and MAE of 0.6852 ± 0.0022, improving significantly upon the strongest baseline, Item-kNN, by 1.75% in RMSE and 1.65% in MAE. Notably, among the baselines, traditional neighborhood-based and matrix factorization methods remain competitive with, and even surpass, more recent neural approaches on this dataset. Adding wishlist information to neural baselines shows inconsistent results, improving NeuMF while degrading DMF.

Table 2. Comparison of our ensemble with existing methods on the holdout set. RMSE and MAE are reported as mean ±95% CI from 10,000 bootstrap resamples. Best result in **bold**.

Method	RMSE	MAE
Naive baselines		
GlobalMean	$1.0455_{\pm 0.0029}$	$0.8474_{\pm 0.0025}$
ItemMean	$0.9989_{\pm 0.0028}$	$0.8046_{\pm 0.0023}$
UserMean	$0.9803_{\pm 0.0029}$	$0.7798_{\pm 0.0024}$
SlopeOne	$0.9249_{\pm 0.0028}$	$0.7317_{\pm 0.0023}$
Neighborhood-based		
KNNBasic	$0.9740_{\pm 0.0032}$	$0.7589_{\pm 0.0025}$
KNNWithZScore	$0.9207_{\pm 0.0029}$	$0.7258_{\pm 0.0023}$
KNNWithMeans	$0.9187_{\pm 0.0028}$	$0.7255_{\pm 0.0023}$
Item-kNN	$0.8900_{\pm 0.0028}$	$0.6967_{\pm 0.0022}$
Matrix factorization		
SVD	$1.1763_{\pm 0.0023}$	$0.9811_{\pm 0.0026}$
IterSVD	$0.9083_{\pm 0.0028}$	$0.7136_{\pm 0.0023}$
ALS	$0.9056_{\pm 0.0029}$	$0.7082_{\pm 0.0023}$
BiasedSVD	$0.8982_{\pm 0.0028}$	$0.7079_{\pm 0.0022}$
SVD++	$0.8910_{\pm 0.0028}$	$0.7007_{\pm 0.0022}$
Neural / linear autoencoder		
EASE	$0.9084_{\pm 0.0028}$	$0.7191_{\pm 0.0022}$
DMF	$0.9031_{\pm 0.0028}$	$0.7176_{\pm 0.0022}$
DMF† (incl. wishlist)	$0.9041_{\pm 0.0028}$	$0.7193_{\pm 0.0022}$
NeuMF	$0.9008_{\pm 0.0028}$	$0.7089_{\pm 0.0022}$
NeuMF† (incl. wishlist)	$0.8977_{\pm 0.0028}$	$0.7053_{\pm 0.0022}$
Our ensemble	$\mathbf{0.8744}_{\pm \mathbf{0.0028}}$	$\mathbf{0.6852}_{\pm \mathbf{0.0022}}$

† Our own extension of the original architecture.

3.1. Ablation experiments

We evaluate how the individual components contribute to the accuracy. The choice of meta-learner is important, with HGBR outperforming Ridge and mean aggregation, suggesting that a non-linear meta-learner captures interactions that a linear meta-learner misses (Figure 2).

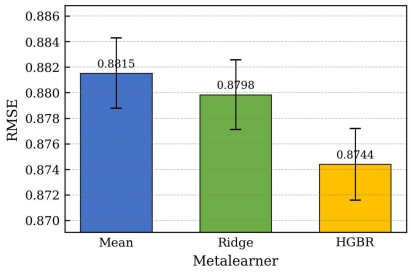


Figure 2. Effect of the meta-learner choice on RMSE. Non-linear (HGBR) stacking outperforms linear stacking (Ridge) and mean aggregation. Mean aggregation excludes the context vector.

Among base predictors, SVD++ and Item-kNN have the largest impact on RMSE, indicating that they are the main contributors to ensemble accuracy (Table 3). Cold-start predictors slightly decrease performance, possibly due to

few cold users or items at 13% rating density.

Table 3. Effect of removing one base predictor class at a time on RMSE. ΔRMSE is the difference relative to the full ensemble.

Model	RMSE	ΔRMSE
Full ensemble	$0.8744_{\pm 0.0028}$	—
w/o SVD++	$0.8808_{\pm 0.0028}$	$+0.0064$
w/o Bagged SVD++	$0.8747_{\pm 0.0028}$	$+0.0003$
w/o EASE / EASEBinary	$0.8757_{\pm 0.0028}$	$+0.0013$
w/o Item-kNN	$0.8779_{\pm 0.0028}$	$+0.0035$
w/o Cold-start base predictors	$0.8740_{\pm 0.0028}$	-0.0004

Increasing ensemble size from 14 to 47 base predictors reduces RMSE monotonically, likely because additional instances per class average out noise in individual predictors (Figure 3).

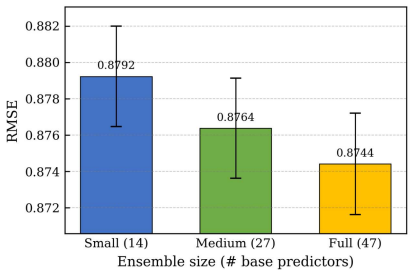


Figure 3. Effect of ensemble size on RMSE. Every size includes at least one predictor from each base predictor class.

Wishlist usage shows a marginal improvement in RMSE, although the effect remains small compared to the variability across bootstrap resamples (Figure 4).

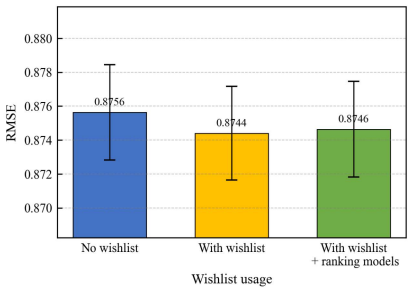


Figure 4. Effect of wishlist usage on RMSE. *No wishlist*: base predictors use ratings only. *With wishlist (proposed)*: base predictors use both ratings and wishlist. *With wishlist+ranking models*: additionally adds ranking-based base predictors trained only on wishlist data.

Contrary to our expectations, the context vector does not reduce RMSE (Figure 5). It may be redundant, as the HGBR meta-learner likely captures user and item statistics implicitly from the base predictor scores.

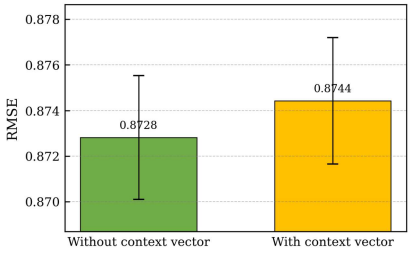


Figure 5. Effect of the context vector on RMSE. Adding the context vector does not improve performance.

4. Discussion and Conclusion

In this work, we introduced a new stacking ensemble with 47 base CF predictors and a hand-crafted context vector of user and item statistics, combined by a gradient-boosted meta-learner, to predict ratings from explicit and implicit feedback. Our results show that the proposed ensemble achieves the lowest RMSE and MAE among all evaluated methods. Ablations indicate that the gain is mainly driven by the non-linear HGBR meta-learner, the contribution of SVD++ and Item-kNN base predictors, and the ensemble size.

Adding wishlist information yielded only a small improvement. Similarly, our attempts to add wishlist information into the NeuMF and DMF baselines yielded inconsistent effects on RMSE. We therefore hypothesize that on the comparatively dense CIL dataset (13% vs. 1% on Netflix) explicit ratings capture most of the predictive signal, making it difficult for the noisier wishlist data to reliably provide additional value.

Overall, our results suggest that the model performance is driven far more by the diversity of the ensemble and the non-linearity of the meta-learner than by the inclusion of supplementary implicit data.

5. Limitations and Future Work

Several limitations merit acknowledgement. First, our experiments were conducted exclusively on the CIL dataset. Compared to established benchmarks such as MovieLens 25M, the CIL dataset is significantly smaller and denser. Therefore, our model needs to be validated on extreme-scale or highly sparse datasets in the future. Second, we have only evaluated the performance of our model using error metrics RMSE and MAE. Other ranking-based metrics, such as NDCG@K and Recall@K, should be considered in future work because they are more common in modern recommender evaluation (Rajesh & Kumar, 2025). Finally, this work motivates a more systematic study of when implicit feedback such as wishlist data adds significant value to rating prediction, and motivates more principled ways of incorporating it.

References

Bell, R. M. and Koren, Y. Lessons from the netflix prize challenge. *SIGKDD Explor. Newsl.*, 9(2):75–79, December 2007. ISSN 1931-0145. doi: 10.1145/1345448.1345465. URL https://doi.org/10.1145/1345448.1345465.

Bennett, J. and Lanning, S. The netflix prize. 2007. URL https://api.semanticscholar.org/CorpusID:9528522.

Breiman, L. Bagging predictors. *Machine Learning*, 24(2): 123–140, August 1996. ISSN 1573-0565. doi: 10.1023/a:1018054314350. URL http://dx.doi.org/10.1023/A:1018054314350.

Ekstrand, M. D., Ludwig, M., Kolb, J., and Riedl, J. T. Lenskit: a modular recommender framework. In *Proceedings of the Fifth ACM Conference on Recommender Systems*, RecSys '11, pp. 349–350, New York, NY, USA, 2011. Association for Computing Machinery. ISBN 9781450306836. doi: 10.1145/2043932.2044001. URL https://doi.org/10.1145/2043932.2044001.

Gomez-Uribe, C. A. and Hunt, N. The netflix recommender system: Algorithms, business value, and innovation. *ACM Trans. Manage. Inf. Syst.*, 6(4), December 2016. ISSN 2158-656X. doi: 10.1145/2843948. URL https://doi.org/10.1145/2843948.

He, X., Liao, L., Zhang, H., Nie, L., Hu, X., and Chua, T.-S. Neural collaborative filtering. In *Proceedings of the 26th International Conference on World Wide Web*, WWW '17, pp. 173–182, Republic and Canton of Geneva, CHE, 2017. International World Wide Web Conferences Steering Committee. ISBN 9781450349130. doi: 10.1145/3038912.3052569. URL https://doi.org/10.1145/3038912.3052569.

Hu, Y., Koren, Y., and Volinsky, C. Collaborative filtering for implicit feedback datasets. In *Proceedings of the 2008 Eighth IEEE International Conference on Data Mining*, ICDM '08, pp. 263–272, USA, 2008. IEEE Computer Society. ISBN 9780769535029. doi: 10.1109/ICDM.2008.22. URL https://doi.org/10.1109/ICDM.2008.22.

Koren, Y. Factorization meets the neighborhood: a multifaceted collaborative filtering model. In *Proceedings of the 14th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '08, pp. 426–434, New York, NY, USA, 2008. Association for Computing Machinery. ISBN 9781605581934. doi: 10.1145/1401890.1401944. URL https://doi.org/10.1145/1401890.1401944.

Koren, Y. and Bell, R. *Advances in Collaborative Filtering*, pp. 145–186. Springer US, Boston, MA, 2011. ISBN 978-0-387-85820-3. doi: 10.1007/978-0-387-85820-3_5. URL https://doi.org/10.1007/978-0-387-85820-3_5.

Koren, Y., Bell, R., and Volinsky, C. Matrix factorization techniques for recommender systems. *Computer*, 42(8): 30–37, August 2009a. ISSN 0018-9162. doi: 10.1109/MC.2009.263. URL https://doi.org/10.1109/MC.2009.263.

Koren, Y., Bell, R., and Volinsky, C. Matrix factorization techniques for recommender systems. *Computer*, 42(8): 30–37, 2009b. doi: 10.1109/MC.2009.263.

Lemire, D. and Maclachlan, A. Slope one predictors for online rating-based collaborative filtering. *ArXiv*, abs/cs/0702144, 2007. URL https://api.semanticscholar.org/CorpusID:2361137.

Rajesh, D. B. and Kumar, A. Collaborative filtering models an experimental and detailed comparative study. *Scientific Reports*, 15(1), August 2025. ISSN 2045-2322. doi: 10.1038/s41598-025-15096-4. URL http://dx.doi.org/10.1038/s41598-025-15096-4.

Rendle, S., Freudenthaler, C., Gantner, Z., and Schmidt-Thieme, L. Bpr: Bayesian personalized ranking from implicit feedback. In *Proceedings of the Twenty-Fifth Conference on Uncertainty in Artificial Intelligence*, UAI '09, pp. 452–461, Arlington, Virginia, USA, 2009. AUAI Press. ISBN 9780974903958.

Sarwar, B., Karypis, G., Konstan, J., and Riedl, J. Item-based collaborative filtering recommendation algorithms. In *Proceedings of the 10th International Conference on World Wide Web*, WWW '01, pp. 285–295, New York, NY, USA, 2001. Association for Computing Machinery. ISBN 1581133480. doi: 10.1145/371920.372071. URL https://doi.org/10.1145/371920.372071.

Smith, B. and Linden, G. Two decades of recommender systems at amazon.com. *IEEE Internet Computing*, 21 (3):12–18, 2017. doi: 10.1109/MIC.2017.72.

Steck, H. Embarrassingly shallow autoencoders for sparse data. In *The World Wide Web Conference*, WWW '19, pp. 3251–3257, New York, NY, USA, 2019. Association for Computing Machinery. ISBN 9781450366748. doi: 10.1145/3308558.3313710. URL https://doi.org/10.1145/3308558.3313710.

Su, X. and Khoshgoftaar, T. M. A survey of collaborative filtering techniques. *Adv. Artif. Intell.*, 2009: 421425:1–421425:19, 2009. URL https://api.semanticscholar.org/CorpusID:1805048.

Xue, H.-J., Dai, X.-Y., Zhang, J., Huang, S., and Chen, J. Deep matrix factorization models for recommender systems. In *Proceedings of the 26th International Joint Conference on Artificial Intelligence*, IJCAI'17, pp. 3203–3209. AAAI Press, 2017. ISBN 9780999241103.

A. Dataset details

Table 4. Statistics of the experimented data.

Dataset	User #	Item #	Rating #	Wishes #	Rating density	Wish density
CIL dataset	9,850	960	1,195,657	345,339	0.1264	0.0365

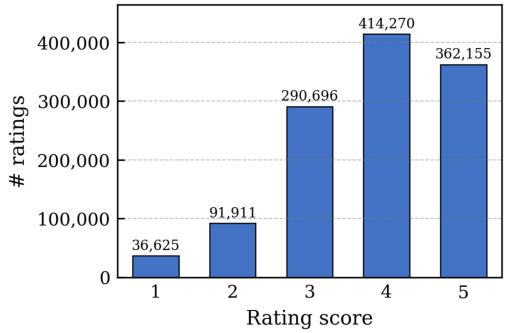


Figure 6. Distribution of paper ratings in the CIL dataset

B. Model details

B.1. Base predictors

Table 5 lists the $N = 47$ base predictors used in the stacking ensemble.

C. Training details

Table 6 lists the hyperparameters for each baseline model. For each model, we selected the best configuration using grid search over all combinations.

D. Evaluation details

Model performance was assessed using the Root Mean Square Error (RMSE) and Mean Absolute Error (MAE) between the observed ratings r_{ui} with predicted ratings $\hat{r}_{ui}$ from users u on items i.

MAE Mean Absolute Error (MAE) measures the average absolute deviation between predicted and true ratings.

$$\text{MAE} = \frac{1}{|R_{\text{test}}|} \sum_{r_{u,i} \in R_{\text{test}}} |\hat{r}_{u,i} - r_{u,i}|$$

RMSE Root Mean Squared Error (RMSE) measures the average prediction error, penalizing larger deviations more strongly by squaring.

$$\text{RMSE} = \sqrt{\frac{1}{|R_{\text{test}}|} \sum_{r_{u,i} \in R_{\text{test}}} (\hat{r}_{u,i} - r_{u,i})^2}$$

D.1. Baseline descriptions

- **GlobalMean**: It predicts the global mean rating.
- **UserMean**: It predicts the user's mean rating.
- **ItemMean**: It predicts the item's mean rating.
- **SlopeOne** (Lemire & Maclachlan, 2007): It predicts ratings using average pairwise rating deviations between items.
- **KNNBasic**: It averages ratings over the k nearest items, weighted by cosine similarity.

- **KNNWithMeans**: It extends KNNBasic by mean-centering each user's ratings.
- **KNNWithZScore**: It extends KNNBasic by z-scoring each user's ratings.
- **Item-kNN** (Sarwar et al., 2001): It predicts ratings as a similarity-weighted sum over the k nearest rated items, using shrunk cosine similarity.
- **SVD**: It computes truncated SVD on a rating matrix where missing values are filled with a default value.
- **IterSVD**: It iteratively imputes missing values using truncated SVD reconstruction.
- **ALS** (Koren et al., 2009b): It learns user and item latent factors by alternating between fixed-user and fixed-item least-squares updates.
- **BiasedSVD** (Koren et al., 2009b): It learns a biased matrix factorisation using gradient descent with global, user, and item bias terms.
- **SVD++** (Koren, 2008): It extends BiasedSVD with implicit feedback by augmenting user factors with a learned sum over interacted items.
- **NeuMF** (He et al., 2017): It fuses a linear matrix factorization pathway with a non-linear MLP to capture complex user-item interactions. To adapt this architecture for explicit matrix completion, we normalize the target ratings to the range $[0, 1]$ and optimize it using continuous binary cross-entropy exclusively on the observed data. During inference, the network's bounded sigmoid predictions are rescaled back to the standard 1–5 rating scale.
- **DMF** (Xue et al., 2017): It employs two parallel MLPs to project full user and item interaction vectors into a shared latent space, predicting preferences using cosine similarity. To adapt this architecture for explicit matrix completion, we construct the network inputs by filling unobserved interactions with zeros, but normalize the target ratings to the range $[0, 1]$ and optimize using continuous binary cross-entropy exclusively on the observed data. During inference, the network's bounded cosine similarity predictions are rescaled back to the standard 1–5 rating scale.
- **EASE** (Steck, 2019): It learns an item-item weight matrix in closed form by reconstructing each user's interactions with a zero-diagonal constraint.

E. Ablation details

E.1. Wishlist usage ablation

The variants compared in Figure 4 are described below.

No wishlist Only the input was modified. The base predictors use ratings only, in particular no wishlist information. The wishlist entries in the context vector were zeroed out. All other components of our ensemble were left unchanged.

Wishlist Our proposed ensemble.

With wishlist+ranking We extended the ensemble with implicit-feedback ranking models from the LensKit library (Ekstrand et al., 2011). These models were trained on the wishlist data only, treating wishlist entries as positive interactions (Table 7).

F. Extended evaluation results

Figure 7 reports the impact of isotonic regression calibration on ensemble RMSE.

G. Computational resources

All experiments were run on the student computing cluster. Each job was allocated 1 CPU core and 1 GPU from a mixed pool (NVIDIA GTX 1080 Ti, RTX 2080 Ti, RTX 5060 Ti, and GB10), with a 24-hour wall-clock limit. Experiments were implemented in PyTorch with CUDA 12.6.

H. Kaggle

For the Kaggle submission, we retrained our model on the full dataset without a held-out test set. Everything else remained the same.

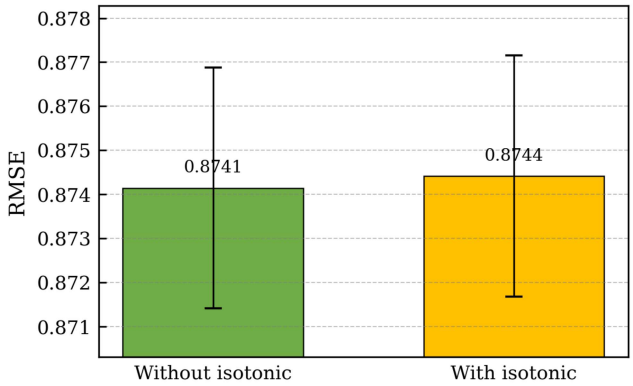


Figure 7. Effect of isotonic regression on RMSE. No measurable effect on the holdout set.

Table 5. The $N = 47$ base predictors used in our ensemble with their initialized hyperparameters. ✓ = used, × = not used.

Model	Hyperparameters	Wishlist	Data normalizer
SVD++			
SVD++	k=25, reg=0.05, seed=42	✓	Identity
SVD++	k=25, reg=0.05, seed=123	✓	Identity
SVD++	k=25, reg=0.05, seed=7	✓	Identity
SVD++	k=25, reg=0.05, seed=31	✓	Identity
SVD++	k=30, reg=0.07, seed=42	✓	Identity
SVD++	k=30, reg=0.07, seed=123	✓	Identity
SVD++	k=25, reg=0.05, seed=42	✓	Scientist
SVD++	k=25, reg=0.05, seed=42	✓	Paper
SVD++	k=25, reg=0.05, seed=11	✓	Identity
SVD++	k=25, reg=0.05, seed=99	✓	Identity
SVD++	k=25, reg=0.05, seed=5	✓	Identity
SVD++	k=20, reg=0.04, seed=42	✓	Identity
SVD++	k=35, reg=0.06, seed=42	✓	Identity
SVD++	k=25, reg=0.05, seed=7	✓	Scientist
SVD++	k=50, reg=0.08, seed=42	✓	Identity
SVD++	k=50, reg=0.08, seed=7	✓	Identity
SVD++	k=35, reg=0.07, seed=42	✓	Paper
SVD++	k=25, reg=0.05, lr_decay=0.95, seed=42	✓	Identity
SVD++	k=25, reg=0.05, lr_decay=0.95, seed=123	✓	Identity
SVD++	k=25, reg=0.05, lr_decay=0.95, seed=7	✓	Identity
SVD++	k=25, reg=0.05, lr_decay=0.95, seed=31	✓	Identity
SVD++	k=25, reg=0.05, lr_decay=0.95, seed=11	✓	Identity
SVD++	k=25, reg=0.03, lr_decay=0.97, seed=42	✓	Identity
SVD++	k=25, reg=0.05, seed=42	×	Identity
SVD++	k=25, reg=0.05, seed=123	×	Identity
SVD++	k=25, reg=0.05, seed=7	×	Identity
SVD++	k=15, reg=0.04, seed=42	✓	Identity
SVD++	k=25, reg=0.05, seed=42	×	Scientist
Bagged SVD++			
Bagged SVD++	k=25, reg=0.05, frac=0.7, seed=101	✓	Identity
Bagged SVD++	k=25, reg=0.05, frac=0.7, seed=202	✓	Identity
Bagged SVD++	k=25, reg=0.05, frac=0.8, seed=303	✓	Identity
Bagged SVD++	k=25, reg=0.05, frac=0.6, seed=404	✓	Identity
Item-kNN			
Item-kNN	k=50, shrinkage=100	×	Identity
Item-kNN	k=30, shrinkage=80	×	Identity
Item-kNN	k=20, shrinkage=50	×	Identity
Item-kNN	k=10, shrinkage=30	×	Identity
Item-kNN	k=60, shrinkage=120	×	Identity
EASE/EASEBinary			
EASE	λ=150	×	Identity
EASE	λ=400	×	Identity
EASEBinary	λ=200	✓	Identity
EASEBinary	λ=500	✓	Identity
EASEBinary	λ=50	✓	Identity
Cold-start			
IterSVD	k=10, n_iters=20	×	Identity
IterSVD	k=15, n_iters=20	×	Identity
SVD	k=3	×	Identity
SVD	k=5	×	Identity
BiasModel	shrink=25	×	Identity

Table 6. Model hyperparameters for baseline models. For each model, we selected the best configuration using grid search over all combinations.

Model	Hyperparameters	Data normalizer
GlobalMean	—	Identity
UserMean	—	Identity
ItemMean	—	Identity
SlopeOne	—	Identity
KNNBasic	k : $[10, 20, 30, 40, 50]$	Identity
KNNWithMeans	k : $[10, 20, 30, 40, 50]$	Identity
KNNWithZScore	k : $[10, 20, 30, 40, 50]$	Identity
Item-kNN	k : $[10, 20, 30, 40, 50]$ shrinkage: $[25.0, 50.0, 100.0]$	Identity, Paper, Scientist
SVD	k : $[2, 5, 10, 20, 40]$ fill: 3.0	Identity, Paper, Scientist
BiasedSVD	n_factors: $[10, 20, 40]$ reg_all: $[0.02, 0.05]$ lr_all: $[0.005, 0.01]$ n_epochs: 20 biased: 1 lr_decay: 1.0 seed: 42	Identity, Paper, Scientist
SVD++	k : $[10, 20, 40]$ reg: $[0.02, 0.05]$ lr: $[0.005, 0.01]$ n_epochs: 20 lr_decay: 1.0 seed: 42	Identity, Paper, Scientist
IterSVD	k : $[5, 10, 20, 40]$ n_iters: $[10, 20]$	Identity, Paper, Scientist
ALS	k : $[5, 10, 20, 40]$ reg: $[0.05, 0.1, 0.5]$ n_iters: 20 seed: 42	Identity, Paper, Scientist
NeuMF	lr: $[0.0001, 0.0005, 0.001, 0.005, 0.01]$ embed_dim: $[8, 16, 32, 64]$ epochs: 10 batch_size: 8192 mlp_layers: $[64, 32, 16, 8]$ use_wishlist: `false` seed: 42	Identity
NeuMF (incl. wishlist)	lr: $[0.0001, 0.0005, 0.001, 0.005, 0.01]$ embed_dim: $[8, 16, 32, 64]$ epochs: 10 batch_size: 8192 mlp_layers: $[64, 32, 16, 8]$ use_wishlist: `true` seed: 42	Identity
DMF	lr: $[0.0001, 0.0005, 0.001, 0.005, 0.01]$ mlp_layers: $[[64, 32, 16], [128, 64, 32]]$ epochs: 20 batch_size: 8192 use_wishlist: `false` seed: 42	Identity
DMF (incl. wishlist)	lr: $[0.0001, 0.0005, 0.001, 0.005, 0.01]$ mlp_layers: $[[64, 32, 16], [128, 64, 32]]$ epochs: 20 batch_size: 8192 use_wishlist: `true` seed: 42	Identity
EASE	lam: $[100.0, 300.0, 500.0, 1000.0]$ shrink: $[10.0, 25.0, 50.0]$ use_bias: [True, False]	Identity

Table 7. Implicit-feedback ranking models with their initialized hyperparameters added to the ensemble for the *With wishlist+ranking models* bar of Figure 4. All models were trained on the wishlist data only.

Model	**Hyperparameters**	**Data normalizer**
BPR (Rendle et al., 2009)	n_factors: 32	Identity
BPR (Rendle et al., 2009)	n_factors: 64	Identity
ItemPop	—	Identity
iALS (Hu et al., 2008)	n_factors: 32	Identity
iALS (Hu et al., 2008)	n_factors: 64	Identity
EASEBinary (Steck, 2019)	λ : 150 use_wishlist: True	Identity